

GNN-EADD: GPU-Accelerated E-Commerce Fraud Detection via Heterogeneous Graphs — Phase 1 Report

Sneha Nagmoti (12342090) Rithika Sannidhi (12341900)
Harshita Patidar (12342920) Snehal Suhane (12342100)

1 Project Overview

This project implements a GPU-accelerated reimplementation and extension of the **GNN-EADD** (Graph Neural Network-based E-Commerce Anomaly Detection via Dual-Stage Learning) framework. The system models the Amazon Electronics marketplace as a **heterogeneous graph** to detect fraudulent products, fake reviews, and shill accounts. The pipeline follows a dual-stage approach: **Stage 1** (unsupervised Graph Autoencoder) learns structural node embeddings, and **Stage 2** (semi-supervised GAT) fine-tunes them using labeled anomalies. Phase 1, documented here, covers the complete data engineering pipeline, anomaly labeling, baseline models, and a fully trained Stage 1 GAE with a custom CUDA kernel.

2 Data Engineering Challenges

The original paper leaves several critical implementation details underspecified. Every challenge below was resolved independently.

2.1 Dataset Selection

No openly available dataset outside Amazon provided sufficient heterogeneous structure (users, products, sellers, reviews). We selected the **Amazon Electronics 5-core** dataset (reviews + meta-data). Multiple yearly snapshots (2014–2023) were tested, as the paper does not specify the version used. After filtering, our graph scale is in the same order of magnitude as reported.

2.2 Seller Node Construction

The dataset contains no explicit seller entity. We engineered seller nodes from the `brand` metadata field. Raw brand strings are highly inconsistent (e.g., "Amazon", "amazon", "AMAZON"). A normalization function lowercases, strips whitespace, and filters an `INVALID_BRANDS` set (unknown, generic, n/a, etc.), producing a deduplicated seller node set.

2.3 ASIN-to-Integer Mapping

Amazon ASINs are alphanumeric strings; GPUs require integer indices. We build a unified, non-overlapping global ID namespace: Users $[0, N_u)$, Products $[N_u, N_u + N_p)$, Sellers $[N_u + N_p, N_u + N_p + N_s)$. The 5-core reviews and metadata files must be cross-linked to the same namespace, requiring careful multi-pass streaming over $\sim 750K$ products.

2.4 Edge Schema Design

The paper describes a heterogeneous graph but provides no rules for edge construction. We independently designed five directed edge types:

Type	Direction	Semantics
E_{pu}	User $\rightarrow$ Product	Purchase/review interaction
E_{up}	Product $\rightarrow$ User	Transpose (bidirectional GCN)
E_{ps}	Product $\rightarrow$ Seller	Listed by brand
E_{sp}	Seller $\rightarrow$ Product	Transpose
E_{uu}	User $\leftrightarrow$ User	Co-purchase proximity

E_{uu} edges propagate signals through collusion networks. Co-buyer sampling is capped at 25 users per product to prevent edge explosion.

2.5 Anomaly Labeling

The paper provides no anomaly labeling methodology or label set. We designed a multi-criteria heuristic pipeline:

- **K-Core Decomposition** — top 0.6% densely connected nodes
- **Review Boosting** — rating ≥ 4.8 , price $< 15\%$ of category median, brand-title mismatch
- **Fake Seller** — `also_buy` > 80 , verified ratio $< 25\%$
- **Temporal Bursts** — > 35 reviews at the same Unix timestamp

Any node meeting one criterion is flagged anomalous.

3 Preprocessing Pipeline

`data_preprocessing.py` runs in four passes designed for 16 GB RAM safety using memory-mapped outputs and chunked NLP encoding.

Pass 1 — ID Space: Enumerates all valid ASINs, users, and brands into a global integer namespace.

Pass 2 — NLP Fraud Signatures: Streams reviews to compute per-node behavioral statistics (review count, rating variance, lexical diversity, VADER sentiment-rating mismatch) without storing raw text.

Pass 3 — Feature Encoding (128-D float16): Products: 96-D text embedding (`all-MiniLM-L6-v2` $\rightarrow$ Incremental PCA) + 24-D category multi-hot + 8-D stats. Users: 112-D category preference + 16-D stats. Sellers: 112-D mean product embedding + 16-D catalog stats.

Pass 4 — CSR Topology: Builds Compressed Sparse Row binaries for all 5 edge types. CSR reduces memory from $\mathcal{O}(N^2)$ to $\mathcal{O}(|E|)$ and enables direct CUDA streaming.

4 Label Generation (`generate_labels.py`)

This script bridges anomaly labeling and Stage 2 training. It (1) maps flagged string IDs to global integer IDs via `node_id_mappings.json`, (2) compiles anomaly positives, (3) samples an equal number of normal nodes (1:1 ratio) to prevent BCE class imbalance, and (4) shuffles and saves

labels.npy of shape $[L, 2]$ (column 0: global ID, column 1: binary label) with fixed seed 42 for reproducibility.

5 Baseline Models

Two baselines provide comparative context.

HeteroDOMINANT (dominant_anomaly.py): Heterogeneous GNN autoencoder using HeteroConv+SAGEConv to minimize $\mathcal{L} = \text{MSE}(X, \hat{X})$; flags anomalies via L_2 reconstruction error using 2.2σ (products) and 4.0σ (buyers) thresholds.

GraphSAGE Autoencoder (sage_anomaly.py): Homogeneous product-to-product graph via also_buy/also_viewed links. Trained with MSE reconstruction loss; L_2 norm scores; dynamic percentile threshold targeting $\sim 4,500$ anomalies.

6 Stage 1: Graph Autoencoder

6.1 Architecture: TripleStreamGAE

A 2-layer heterogeneous GCN encoder with type-specific weight matrices and three parallel CUDA streams. The layer update follows:

$$H_u^{(l)} = \text{ReLU} \left(\sum_{r \in \{pu, uu\}} W_r^{(l)} \tilde{A}_r X + W_{\text{self}, u}^{(l)} X_u \right)$$

where $\tilde{A} = D^{-1/2} A D^{-1/2}$ is symmetric GCN normalization computed globally across all node types. Product and seller nodes follow analogous equations with their respective relation sets.

6.2 Training

Hyperparameter	Value
Dimensions (in/hidden/out)	128 / 128 / 128
Epochs	200
Optimizer / LR	Adam / 0.001
Loss	BCE with logits (link prediction)
Positive samples/epoch	30K per edge type (90K total)
Precision	FP16 forward, FP32 decoder & loss

Key decisions: (1) Self-loops are used in CSR for GCN normalization but masked out of the link prediction loss ($\text{src} \neq \text{dst}$). (2) The dot-product decoder runs outside autocast in FP32 to prevent overflow. (3) GradScaler prevents FP16 gradient underflow. Output: Z_embeddings_stage1.npy of shape $(N_{\text{total}} \times 128)$.

7 Custom CUDA Extension (cuda_spmm/)

A custom warp-aligned CUDA/C++ extension (spmm_kernel.cu) implements two kernels.

Encoder — csr_spmm_warp_kernel: One warp (32 threads) processes one node. Supports dynamic feature dimensions via chunked accumulation (up to 32 chunks of 32 lanes). Reads FP16 features and accumulates in FP32. Pre-computed symmetric edge weights $D_{ij}^{-1/2}$ are applied per neighbor.

Decoder — warp_sparse_dot_product_kernel: One warp computes one edge’s dot product $Z_{\text{src}} \cdot Z_{\text{dst}}$ via __shfl_down_sync warp-shuffle reduction, eliminating global synchronization.

Backprop: A custom torch.autograd.Function performs chunk-streaming gradient accumulation (5K rows/chunk) to prevent VRAM spikes over 1.5M nodes.

8 Preprocessing Results

Metric	Value
Users $ V_u $	728,719
Products $ V_p $	756,077
Sellers $ V_s $	55,679
Total Nodes	1,540,475
E_{pu} User $\rightarrow$ Product	7,253,058
E_{uu} User $\leftrightarrow$ User	42,645,547
Total Edges	~ 59.4 Million
GPU VRAM (features)	~ 0.39 GB
Processing Time	52.7 minutes

9 Phase 2 Plan

Stage 2 will load Z_embeddings_stage1.npy and labels.npy to fine-tune a semi-supervised GAT. A hardware-level attention kernel using __shfl_sync warp primitives will perform all-neighbor reductions with numerically stable softmax, eliminating global barriers.

10 Key Takeaways

Phase 1 demonstrated that faithfully reimplementing a research paper often demands as much — if not more — effort in data engineering as in model design. The GNN-EADD paper provides a compelling model architecture, yet omits the precise mechanisms required to reproduce it: the exact dataset version is never cited, the heterogeneous edge schema is described only at an abstract level, and the anomaly labeling methodology used in Stage 2 is entirely absent.

Each gap required careful, independent decision-making. Constructing seller nodes from inconsistent brand strings, building a unified integer ID namespace across two large disjoint files, designing a five-relation edge schema, and devising a multi-criteria anomaly labeling strategy — all at the scale of 1.54 million nodes and ~ 60 million edges — formed the true engineering core of this phase.

Our implementation goes beyond the paper in several respects: the custom warp-aligned CUDA SpMM kernel directly replaces generic PyTorch sparse operations with hardware-optimised code; the chunk-streaming backpropagation prevents VRAM spikes that would otherwise make training infeasible on consumer GPUs; and the balanced label generation script ensures that Stage 2 training is not biased by the extreme rarity of anomalous nodes. Together, these contributions represent a production-grade, GPU-first reimplement that is both faithful to the paper’s intent and independently validated through two baseline models.